UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Programa educativo: Ingeniería en Tecnologías de la Información e Innovación Digital

Asignatura: Fundamentos de Inteligencia Artificial

Actividad: 6. Algoritmos genéticos

Título del proyecto: Optimización de rutas mediante un algoritmo genético en Python

Integrantes:

- José Manuel Arizmendi Varela
- Erica Azucena Pérez Rivera

Docente: Mtro. Javier García Lara

Grupo: 172

Cuatrimestre: Séptimo

1. INTRODUCCIÓN

Los algoritmos genéticos son técnicas de inteligencia artificial inspiradas en el proceso de evolución natural. Su funcionamiento se basa en generar posibles soluciones a un problema y mejorarlas progresivamente mediante procesos de selección, cruce y mutación.

Estos algoritmos se utilizan para resolver problemas de optimización en los que existen muchas soluciones posibles y resulta complicado encontrar la mejor mediante una búsqueda exhaustiva.

En este proyecto se implementará un algoritmo genético en Python utilizando la biblioteca DEAP. El problema seleccionado será el del viajante, conocido como TSP por sus siglas en inglés. Consiste en encontrar una ruta que permita visitar un conjunto de ciudades, regresar al punto de partida y minimizar la distancia total recorrida.

El programa generará una población de rutas, evaluará la distancia de cada una y aplicará operaciones genéticas durante 30 generaciones. Finalmente, se mostrarán los resultados mediante gráficas para analizar cómo evoluciona la mejor solución encontrada.

2. PLANTEAMIENTO DEL PROBLEMA

Cuando se necesita visitar varias ciudades, existen diferentes maneras de organizar el recorrido. Algunas rutas requieren recorrer una distancia pequeña, mientras que otras pueden ser mucho más largas.

Encontrar la mejor combinación de ciudades puede resultar complicado cuando aumenta la cantidad de lugares que se deben visitar, porque el número de rutas posibles crece rápidamente.

Por este motivo, se propone desarrollar un programa que utilice un algoritmo genético para buscar una ruta de menor distancia entre varias ciudades.

El programa generará distintas rutas y evaluará la distancia total de cada una. Después, seleccionará las soluciones más convenientes y aplicará operaciones de cruce y mutación para producir nuevas rutas.

El proceso se repetirá durante 30 generaciones y se registrará la distancia de la mejor solución encontrada en cada generación.

2.1 Pregunta del problema

¿Cómo se puede utilizar un algoritmo genético para encontrar una ruta de menor distancia entre varias ciudades mediante procesos de selección, cruce y mutación?

2.2 Descripción de la solución

Se utilizará una red de ciudades representadas mediante coordenadas en un plano. La distancia entre cada par de ciudades se calculará mediante la distancia euclidiana.

Cada posible solución estará representada por una lista ordenada de ciudades. Esta lista se conocerá como cromosoma y cada ciudad que forma parte de ella representará un gen.

El algoritmo evaluará la distancia total de cada recorrido y buscará minimizarla mediante la evolución de la población durante 30 generaciones.

3. JUSTIFICACIÓN

Los algoritmos genéticos son importantes en inteligencia artificial porque permiten explorar espacios de soluciones complejos mediante un proceso inspirado en la evolución natural.

El problema del viajante es un ejemplo útil para comprender esta técnica, ya que existen numerosas formas de organizar una ruta y cada combinación puede producir una distancia diferente.

La implementación del algoritmo permitirá comprender cómo se representa una solución, cómo se evalúa su calidad y cómo las operaciones genéticas contribuyen a generar nuevas alternativas.

Además, el uso de Python y DEAP facilitará la construcción del programa y la obtención de resultados que podrán analizarse mediante gráficas.

Este proyecto permitirá reforzar conocimientos de programación, optimización, estructuras de datos y algoritmos de inteligencia artificial.

4. OBJETIVOS

4.1 Objetivo general

Implementar un algoritmo genético en Python utilizando la biblioteca DEAP para buscar una ruta de menor distancia en el problema del viajante y analizar su evolución durante 30 generaciones.

4.2 Objetivos específicos

- Representar un conjunto de ciudades mediante coordenadas.
- Crear una población inicial de rutas posibles.
- Definir una función que calcule la distancia total de cada recorrido.
- Implementar los procesos de selección, cruce y mutación.
- Ejecutar el algoritmo durante 30 generaciones.
- Registrar la mejor distancia encontrada en cada generación.
- Generar gráficas para visualizar la evolución de las soluciones.
- Documentar el procedimiento para facilitar su reproducción.

5. MARCO TEÓRICO

5.1 Inteligencia artificial

La inteligencia artificial es un área de la informática que desarrolla sistemas capaces de resolver problemas, analizar información y encontrar soluciones mediante diferentes técnicas.

Los algoritmos genéticos forman parte de las técnicas de optimización que pueden utilizarse para buscar soluciones en problemas con numerosas alternativas.

5.2 Algoritmos genéticos

Un algoritmo genético es una técnica de búsqueda y optimización inspirada en la selección natural.

Su funcionamiento comienza con una población de soluciones posibles. Cada solución se evalúa y las más convenientes pueden tener mayores posibilidades de participar en la creación de nuevas soluciones.

Mediante diferentes operaciones, el algoritmo genera nuevas alternativas y conserva las mejores soluciones encontradas.

5.3 Población

La población es el conjunto de soluciones que el algoritmo analiza en una generación.

En este proyecto, cada individuo representa una ruta posible para visitar todas las ciudades.

Una población con diferentes rutas permite explorar distintas alternativas para resolver el problema.

5.4 Cromosoma y genes

Un cromosoma es la representación de una solución dentro del algoritmo genético.

En el problema del viajante, el cromosoma será una lista ordenada de ciudades.

Cada posición de la lista representará un gen, que corresponde a una ciudad incluida en el recorrido.

Por ejemplo, una ruta puede representarse de la siguiente manera:

Ciudad A → Ciudad C → Ciudad B → Ciudad D → Ciudad A.

El punto de partida se utiliza también como destino final para cerrar el recorrido.

5.5 Función de aptitud o fitness

La función de aptitud permite evaluar qué tan buena es una solución para el problema.

En este proyecto, la aptitud estará relacionada con la distancia total recorrida. Como se desea encontrar una ruta corta, el objetivo será minimizar esa distancia.

Una ruta con una distancia menor será más conveniente que otra con una distancia mayor, siempre que ambas visiten todas las ciudades requeridas.

5.6 Selección

La selección determina qué individuos pueden participar en la generación de nuevas soluciones.

En este proyecto se utilizará selección por torneo. El procedimiento consiste en elegir varios individuos al azar y seleccionar el más conveniente de entre ellos.

Esta operación favorece la participación de las soluciones con mejor aptitud.

5.7 Cruce

El cruce combina información de dos soluciones para generar nuevas alternativas.

Como las rutas contienen ciudades que no deben repetirse, se utilizará un operador de cruce apropiado para permutaciones, de manera que se mantenga la validez de los recorridos.

El objetivo es combinar características de distintas rutas para obtener posibles soluciones mejores.

5.8 Mutación

La mutación introduce pequeños cambios en una solución.

En este proyecto se intercambiarán las posiciones de dos ciudades seleccionadas dentro de una ruta.

Esta operación permite explorar nuevas combinaciones y ayuda a evitar que la población se mantenga siempre en soluciones muy similares.

5.9 Generaciones y convergencia

Una generación corresponde a una etapa del algoritmo en la que se evalúa una población y se producen nuevas soluciones.

Durante varias generaciones, el algoritmo busca mejorar la calidad de las rutas.

La convergencia se puede analizar mediante una gráfica que muestre cómo cambia la mejor distancia encontrada a lo largo de las generaciones.

Una disminución de la distancia indica que se han encontrado recorridos más cortos.

Sin embargo, encontrar una solución mejor no garantiza que se haya alcanzado la ruta óptima global.

5.10 Biblioteca DEAP

DEAP es una biblioteca de Python diseñada para desarrollar algoritmos evolutivos.

Permite definir individuos, poblaciones, funciones de evaluación y operadores genéticos como selección, cruce y mutación.

En este proyecto se utilizará para implementar el algoritmo genético y facilitar el registro de los resultados de cada generación.

6. DESARROLLO DEL PROYECTO

6.1 Herramientas utilizadas

Para desarrollar el proyecto se utilizarán las siguientes herramientas:

- **Python:** lenguaje de programación principal.
- **DEAP:** biblioteca para implementar el algoritmo genético.
- **Matplotlib:** biblioteca para crear las gráficas de resultados.
- **Visual Studio Code o IDLE:** entorno para escribir y ejecutar el programa.
- **GitHub:** plataforma para almacenar el código y las instrucciones de ejecución.

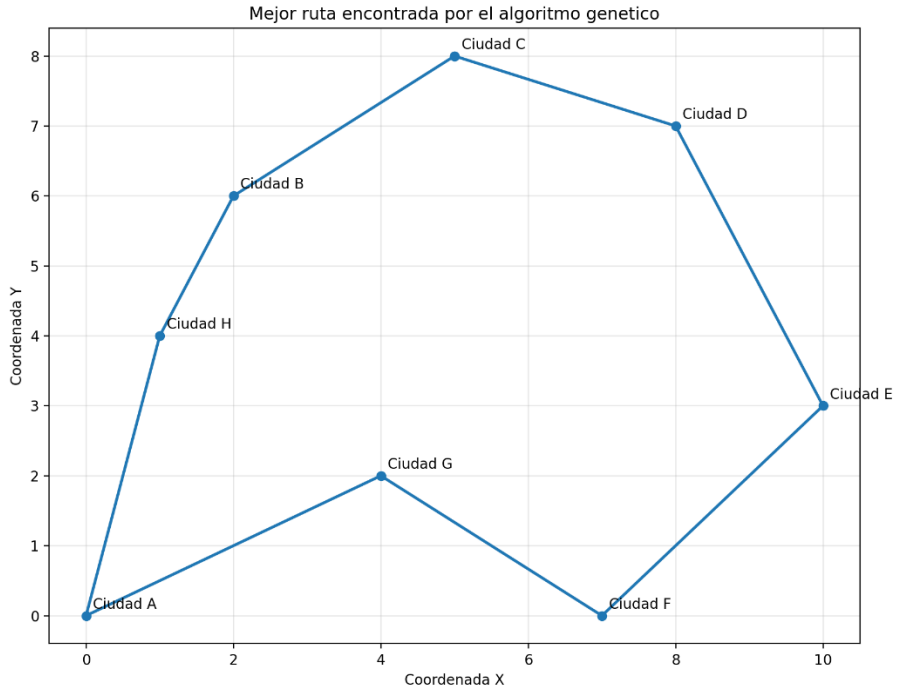
6.2 Representación del problema

Las ciudades se representarán mediante coordenadas en un plano. Cada ciudad tendrá una posición definida por sus valores X e Y.

La distancia entre dos ciudades se calculará mediante la siguiente fórmula:

$$d = \sqrt{((x_2 - x_1)^2 + (y_2 - y_1)^2)}$$

La distancia total de una ruta será la suma de las distancias entre ciudades consecutivas, incluyendo el recorrido desde la última ciudad hasta la primera.



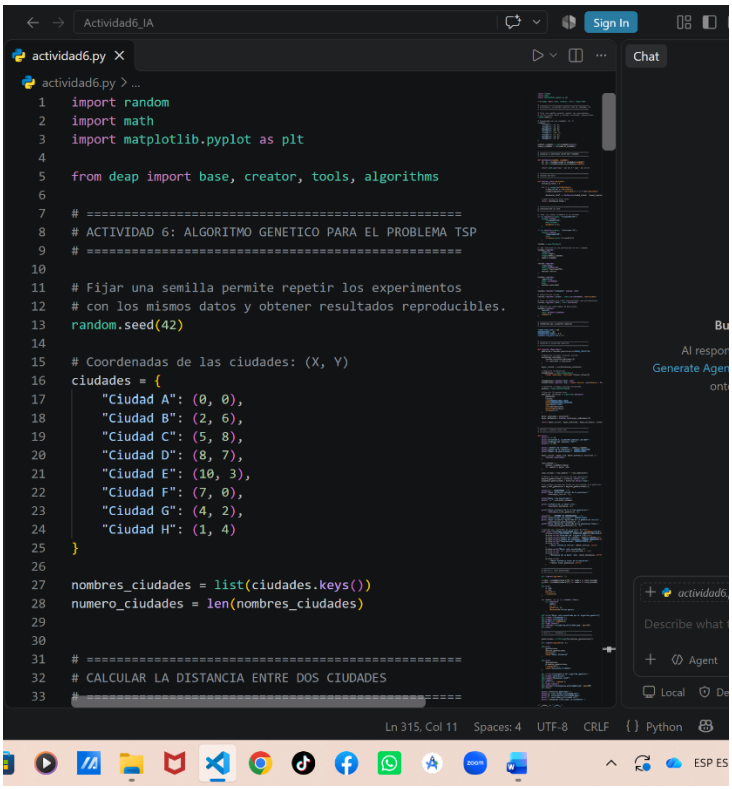
Fuente: elaboración propia.

6.3 Implementación del algoritmo genético

El programa comenzará creando una población inicial de rutas. Posteriormente, calculará la distancia de cada individuo y seleccionará soluciones para reproducirlas.

Durante el proceso se aplicarán operadores de cruce y mutación. Después, se evaluarán las nuevas soluciones y se conservarán las alternativas más convenientes de acuerdo con el procedimiento implementado.

El algoritmo se ejecutará durante 30 generaciones y registrará la mejor distancia encontrada en cada una.



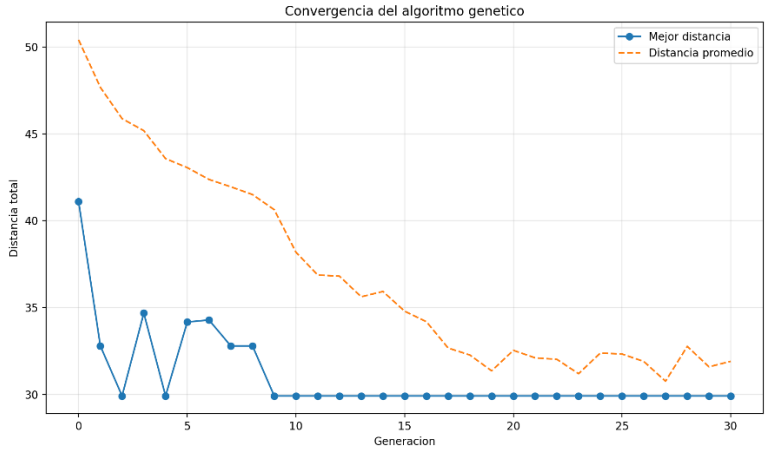
Fuente: elaboración propia.

6.4 Visualización de resultados

Se generarán gráficas para facilitar la interpretación de los resultados.

La primera gráfica mostrará la mejor ruta encontrada entre las ciudades. La segunda mostrará la evolución de la mejor distancia durante las 30 generaciones.

Estas visualizaciones permitirán observar la solución obtenida y analizar si el algoritmo encontró recorridos más cortos conforme avanzó la ejecución.



Fuente: elaboración propia.

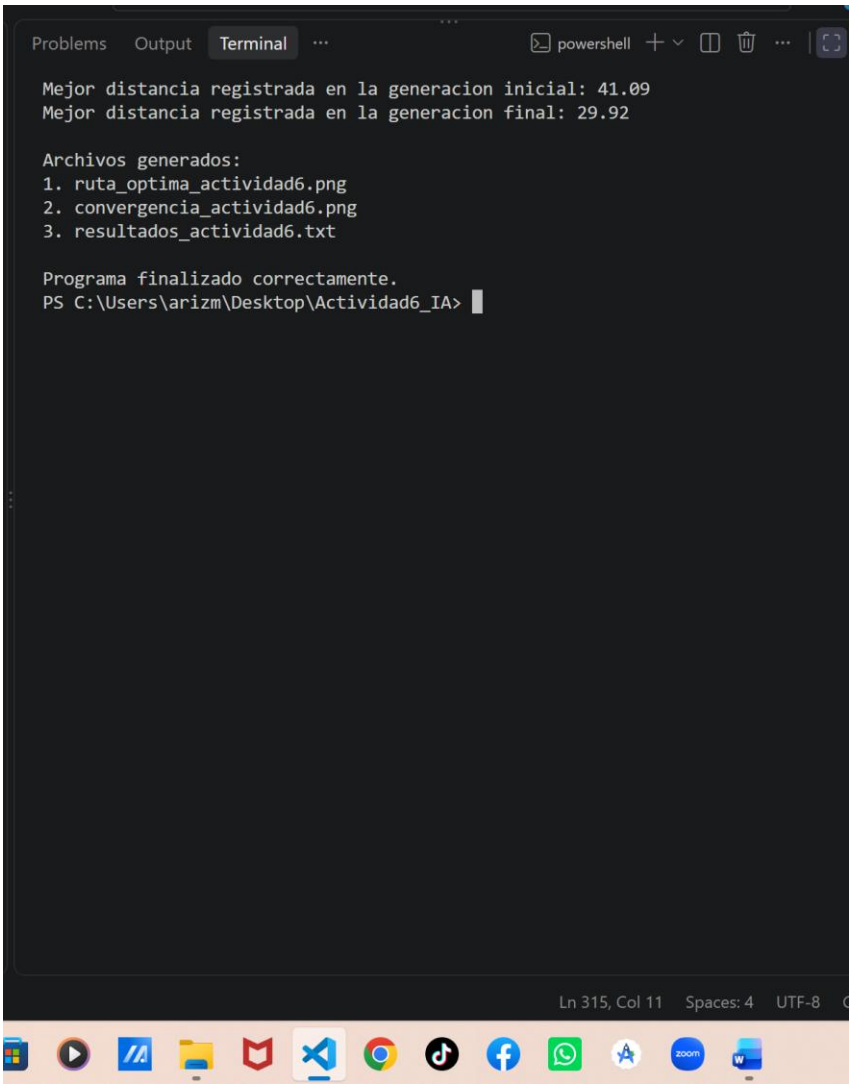
7. PRUEBAS Y RESULTADOS

7.1 Procedimiento de prueba

Para comprobar el funcionamiento del programa, se ejecutará el algoritmo genético con una población de rutas posibles y un total de 30 generaciones.

En cada generación se evaluarán las rutas y se registrará la mejor distancia encontrada.

Al finalizar, el programa mostrará la ruta seleccionada, su distancia total y el historial de evolución del algoritmo.



Fuente: elaboración propia.

7.2 Tabla de resultados

Completar la tabla después de ejecutar el programa.

Indicador	**Resultado**
Número de ciudades	8
Tamaño de la población	100
Número de generaciones	30
Mejor ruta encontrada	Ciudad c – ciudad b-ciudadh- ciudad a- ciudad g- ciudad f
Distancia total de la mejor ruta	29.92
Mejor distancia inicial	41.09
Mejor distancia final	29.92

7.3 Análisis de los resultados

Los resultados permitirán observar el comportamiento del algoritmo genético durante el proceso de optimización.

Se comparará la mejor distancia de las primeras generaciones con la distancia obtenida al finalizar la ejecución.

Si la distancia disminuye, se podrá observar que el algoritmo encontró una ruta más corta durante el proceso de búsqueda. Si la distancia se mantiene durante varias generaciones, esto puede indicar que la población no encontró una solución mejor en ese intervalo.

Es importante considerar que los algoritmos genéticos son métodos de búsqueda heurística. Por ello, la mejor ruta encontrada no necesariamente será la solución óptima global.

El análisis final se realizará con los datos reales generados por el programa.

8. APLICACIÓN EN UN CASO EMPRESARIAL O SOCIAL

El problema del viajante puede relacionarse con actividades de distribución y transporte.

Por ejemplo, una empresa que entrega productos en diferentes ubicaciones necesita organizar sus recorridos para reducir la distancia total recorrida.

Un algoritmo genético puede ayudar a generar y evaluar diferentes órdenes de visita, con el propósito de encontrar una ruta de menor distancia.

Una aplicación similar puede utilizarse para planificar visitas de personal técnico, recorridos de mantenimiento o entregas de productos.

En este proyecto se utilizará una red de ciudades de ejemplo. Por lo tanto, los resultados demostrarán el funcionamiento del algoritmo en un escenario simulado y no representarán mediciones reales de una empresa.

9. CONCLUSIONES

El desarrollo de este proyecto permitirá comprender cómo los algoritmos genéticos pueden utilizarse para resolver problemas de optimización.

Mediante Python y la biblioteca DEAP se implementará un sistema capaz de generar rutas posibles, evaluarlas y mejorarlas utilizando selección, cruce y mutación.

La ejecución durante 30 generaciones permitirá observar los cambios en la mejor solución encontrada y analizar el comportamiento del algoritmo mediante gráficas.

El problema del viajante representa un ejemplo práctico de cómo las técnicas de inteligencia artificial pueden apoyar la planificación de recorridos y la búsqueda de soluciones eficientes.

Finalmente, esta actividad reforzará los conocimientos de programación, optimización y algoritmos evolutivos. También permitirá comprender que el resultado de un algoritmo genético depende de factores como la población inicial, los operadores utilizados y los parámetros de ejecución.

Nota: al finalizar las pruebas, agregar una observación específica sobre la distancia obtenida y el comportamiento de la gráfica de convergencia.

10. REFERENCIAS

Deap Development Team. (s. f.). *DEAP documentation*. https://deap.readthedocs.io/en/master/

Python Software Foundation. (s. f.). *Python documentation*. https://docs.python.org/3/

Matplotlib Development Team. (s. f.). *Matplotlib documentation*. https://matplotlib.org/stable/

Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

11. DECLARACIÓN DEL USO DE INTELIGENCIA ARTIFICIAL

Se utilizó una herramienta de inteligencia artificial como apoyo para comprender el funcionamiento de los algoritmos genéticos, organizar la documentación y orientar el desarrollo del programa en Python. El código se ejecutará para verificar su funcionamiento y obtener los resultados que se presentarán en el reporte. Los integrantes revisarán el procedimiento, comprobarán los resultados y realizarán los ajustes necesarios antes de entregar el trabajo.